

Bloom as built: what the system does today

A code-level account of how Bloom learning paths are generated, stored, shown and progressed. It describes, it does not recommend. Every statement cites a file so it can be checked.

Branch **feat/youth-idealab-foundation** at **375f5a6** plus working tree Written **5 Oct 2026**

Paths are relative to the repo root

CONFIRMED read directly in code or schema **INFERRED** reasoned from code, not observed

UNAVAILABLE could not be checked

Scope limit. The live database was not read: a read-only query of the hosted Supabase project was blocked by this session's permission rules. Everything about the example path "How a Small Business Really Works" therefore comes from the code plus the details given in the request, and is marked as such in section 12.

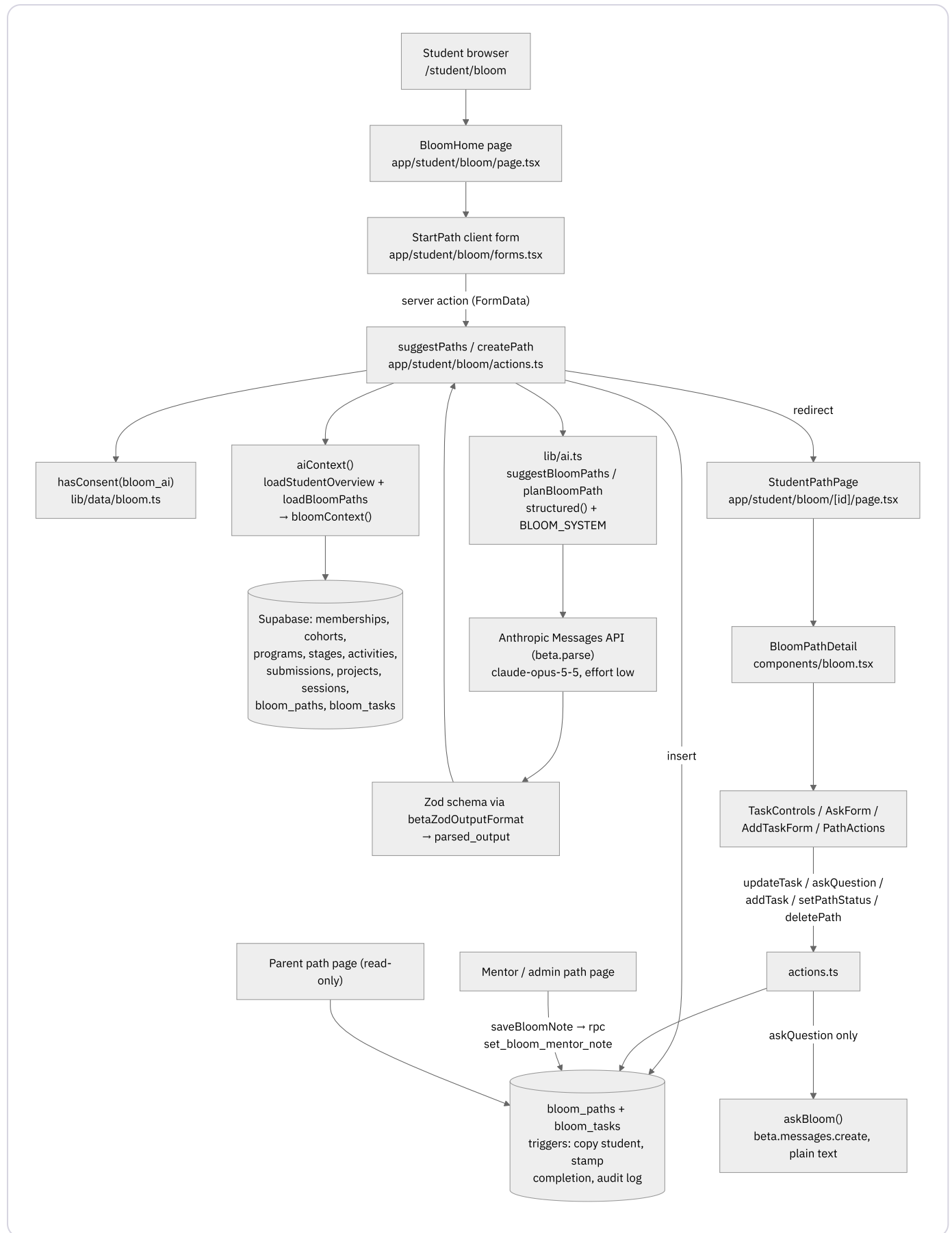
The working tree has one uncommitted UI change from earlier today: Bloom buttons now use `apps/web/src/components/ui/SubmitButton.tsx` for their pending state. It doesn't change any behaviour described here.

01 Executive summary

Bloom is a per-student, student-initiated space for short learning paths. A path is a title, a goal, an AI-written summary and an ordered list of 3, 5 or 7 tasks. Each task is typed *learn*, *do* or *reflect*. **CONFIRMED**

- **No API routes.** Everything runs through Next.js server actions in `apps/web/src/app/student/bloom/actions.ts` that talk to Supabase (Postgres with row-level security). There are no REST endpoints, background jobs or queues.
- **Up to three AI functions, all in** `apps/web/src/lib/ai.ts` . `suggestBloomPaths` returns 3 path ideas as JSON. `planBloomPath` returns a summary and tasks as JSON. `askBloom` returns a plain-text answer. All use Anthropic model `claude-opus-5-5` at low effort, and prompts are hard-coded TypeScript strings.
- **What the AI is told about the student.** Age group, current programme step name, week number, the project's area, title, problem and status, and up to 12 earlier path titles. Nothing else from the student's work, reflections, questions, mentor notes or course submissions is sent.
- **Progress is self-reported.** A task moves through `todo` → `doing` → `done` when the student clicks. No evidence, check or approval is needed. When every task is done the path becomes `completed` ; reopening a task sets it back to `active` .
- **One free-text field per task.** `bloom_tasks.reflection` stores both "Add a note" and reflect-task answers. Questions to Bloom and the answers are kept in `bloom_questions` .
- **Others can watch.** Mentors, parents and admins can read everything through RLS (`can_view_student`). Mentors and admins can leave one note per path. AI features need a parent's `bloom_ai` consent.
- **Not linked to the course.** A path is tagged with the cohort and programme step it started in, but it has no link to the project row, programme activities or submissions.

02 Current architecture



BOX	FILE	MAIN SYMBOL	RESPONSIBILITY
Bloom home	apps/web/src/app/student/bloom/page.tsx	BloomHome	Loads paths and consent, shows stats and the "Start something new" card, and lists active, completed and archived paths.
Start form	apps/web/src/app/student/bloom/forms.tsx	StartPath	Interest field and "Suggest ideas" button, suggestion cards with "Start this path", and the own-topic form (title, goal, depth, "Let Bloom plan").
Server actions	apps/web/src/app/student/bloom/actions.ts	suggestPaths , createPath , addTask , updateTask , setPathStatus , deletePath , askQuestion	Every Bloom write. Each checks <code>role === "student"</code> . AI actions also check <code>bloom_ai</code> consent.
Context builder	apps/web/src/lib/data/bloom.ts L65–77	bloomContext	Turns the course overview and path titles into the <code>BloomContext</code> object sent to the AI.
Course overview	apps/web/src/lib/data/overview.ts L13–80	loadStudentOverview	Most recent student membership, its cohort, programme, stages, activities, submissions, project, sessions and current week.
Bloom reads	apps/web/src/lib/data/bloom.ts	loadBloomPaths , loadBloomPath , hasConsent , bloomStats , loadTimeline , loadLastActive , loadBloomTotals	Reads and derived counts (done and total per path).
AI service and prompts	apps/web/src/lib/ai.ts	BLOOM_SYSTEM , structured , suggestBloomPaths , planBloomPath , askBloom	Prompt text, Zod schemas, Anthropic calls and the mapping of errors to reasons.
Renderer	apps/web/src/components/bloom.tsx	BloomPathCard , BloomPathDetail , QuestionList , BloomPathList , Timeline	Server components shared by the student, mentor, parent and admin views. Role-specific controls are passed in as slots.

BOX	FILE	MAIN SYMBOL	RESPONSIBILITY
Path page	apps/web/src/app/student/bloom/[id]/page.tsx	StudentPathPage	Path header, the detail view with task controls and Ask forms, and "Add your own step".
Schema	supabase/migrations/20261005140000_bloom.sql	3 tables, 3 enums, triggers, RLS, set_bloom_mentor_note	Storage, ownership and visibility rules, completion timestamps, audit log, data export.
Mentor note	apps/web/src/app/mentor/actions.ts L173–185, mentor/forms.ts x BloomNoteForm	saveBloomNote	Mentor or admin note on a path through a security-definer RPC.
Consent	apps/web/src/app/parent/actions.ts L114–140	setConsent	A parent grants or revokes bloom_ai .

State management: React 19 `useActionState` plus local `useState` only, with no client store. Suggestions live only in the `suggestPaths` action state and are never saved. Pages are server components that re-read Supabase after each action's `refresh()` . **CONFIRMED**

03 Bloom generation flow

There are two entry routes into the same `createPath` action.

Route A: "Suggest ideas", then "Start this path"

Two AI calls. The first returns 3 ideas, which are not saved. The second plans the chosen one. The card always submits `depth=standard` (5 tasks) and `useAi=on` .

Route B: "I have my own topic"

One AI call, or none if "Let Bloom plan" is unticked. The student supplies the title (required), goal (optional) and depth (3, 5 or 7 tasks).

USER ACTION

Route A: types an optional interest (max 300 characters) and clicks **Suggest ideas**, then **Start this path** on a card. Route B: fills the own-topic form and clicks **Create path**.

FRONTEND

`StartPath` in `forms.tsx` . These are native `<form action={...}>` elements bound to `useActionState(suggestPaths)` and `useActionState(createPath)` . No fetch code.



The 20 questions, answered

#	QUESTION	ANSWER
1	Trigger	"Suggest ideas", then "Start this path"; or "Create path" with "Let Bloom plan" ticked. Bloom never generates a path on its own.
2	Frontend function	<code>StartPath</code> forms bound to <code>suggest</code> and <code>create</code> from <code>useActionState</code> .
3	Endpoint	Server actions <code>suggestPaths</code> (L40) and <code>createPath</code> (L51) in <code>app/student/bloom/actions.ts</code> . No route handler.
4	Data sent	FormData: <code>interest</code> , or <code>title</code> , <code>goal</code> , <code>depth</code> and <code>useAi</code> . The identity comes from the session cookie.
5	Student and project context	<code>BloomContext</code> : <code>ageGroup</code> , <code>step</code> (stage name), <code>week</code> , <code>project {area, title, problem, status}</code> , <code>previousPaths</code> .

#	QUESTION	ANSWER
6	Previous Bloom history?	Titles only, up to 12, newest first. All statuses are included, archived too.
7	Project description?	Yes: <code>projects.title</code> and <code>projects.problem</code> , plus area and status, for the student's most recent cohort.
8	Previous paths?	Titles only. No goals, summaries or tasks.
9	Completed steps?	No.
10	Mentor or student notes?	No. Not <code>mentor_note</code> , <code>reflection</code> , questions, course feedback or progress cards.
11–12	Prompt and where	Quoted verbatim in section 5. Hard-coded in <code>apps/web/src/lib/ai.ts</code> .
13	Provider and model	Anthropic, <code>@anthropic-ai/sdk ^0.131.0</code> , <code>MODEL = "claude-opus-5-5"</code> (ai.ts L10). With <code>fallbacks: "default"</code> the API may serve the request from a fallback model. INFERRED from the parameter name; the response's model field is not logged.
14	One or many calls	Route A: 2 calls (suggest, then plan). Route B: 1 call, or 0 without AI. Each call is a single turn.
15	What the model generates	The whole plan in one response: <code>summary</code> plus <code>tasks[]</code> , each with <code>kind</code> , <code>title</code> and <code>details</code> . In route A the path <i>title</i> and <i>goal</i> also come from AI, from the suggestion call. There are no reflection questions as a separate structure; the reflect task's <code>details</code> text holds them. If the goal is blank, the prompt asks the model to infer one, but the schema has no goal field, so the inferred goal is never saved.
16–17	Response format	Structured JSON through the API's output format, built from Zod. <code>askBloom</code> returns plain text.
18	Validation	Zod structure only: field presence, string types and the kind enum. Counts ("exactly 3", "exactly 5"), lengths ("under 70 characters", "60–180 words") and ordering are only <code>.describe()</code> hints or prompt text, so they aren't enforced. Code then truncates: suggestions <code>slice(0,3)</code> , tasks <code>slice(0,10)</code> , title 160 characters, details 6000. Database checks require title length 1–160, details ≤6000, goal ≤1000, and the enum kinds.

#	QUESTION	ANSWER
19	Malformed output	The action returns <code>{status:"error", message:"ai.failed"}</code> and nothing is saved. If the path insert succeeds but the task insert fails (for example an empty task title breaking the check), the error is only logged (L101). The student is still redirected to a path with no tasks, and its summary may already be saved.
20	Persistence	<code>bloom_paths</code> gets <code>student_id</code> , <code>cohort_id</code> , <code>title</code> , <code>goal</code> , <code>depth</code> , <code>stage_key</code> (programme step for the current week), <code>summary</code> and <code>ai_generated = true</code> when planned. Then <code>bloom_tasks</code> rows are inserted. Triggers copy <code>student_id</code> from the path, stamp <code>completed_at</code> and write the audit log.

04 Exact AI integration

FUNCTION	SDK CALL	OUTPUT	MAX_TOKENS	CALLER
<code>suggestBloomPaths</code>	<code>beta.messages.parse through structured()</code>	JSON <code>{suggestions: [{title,goal,why}]}</code>	8000	<code>suggestPaths</code>
<code>planBloomPath</code>	<code>beta.messages.parse through structured()</code>	JSON <code>{summary, tasks: [{kind,title,details}]}</code>	8000	<code>createPath</code>
<code>askBloom</code>	<code>beta.messages.create</code>	Plain text, text blocks joined	4000	<code>askQuestion</code>
<code>draftProgressCard</code> (not Bloom)	<code>beta.messages.create</code>	Plain text	4000	mentor <code>draftCardWithAi</code>

- Common settings: `model: "claude-opus-5-5"`, `betas: ["server-side-fallback-2026-07-01"]`, `fallbacks: "default"`, `output_config.effort: "low"`. No temperature, tools, streaming, prompt caching or conversation history.
- The key comes from `ANTHROPIC_API_KEY`. `aiConfigured()` only checks that it is set. The Bloom pages hide AI controls unless `consent && aiConfigured()`.
- Logging: only failures, through `console.error`. Prompts, responses, token usage and the model actually used are **not** stored anywhere. **CONFIRMED**
- There is no app-level rate limiting or usage cap per student. Only the provider's 429 errors are handled. **CONFIRMED**

05 Existing prompts, verbatim

All prompts are hard-coded template strings in `apps/web/src/lib/ai.ts`. None are stored in the database, config files or environment variables. There is **no** separate reflection, follow-up or "regenerate or change path" prompt. **CONFIRMED**

5.1 System prompt BLOOM_SYSTEM (shared by all three Bloom calls)

apps/web/src/lib/ai.ts L90-94

You are Bloom, the learning guide inside Youth Idea Lab, an 8-week live online course where students aged 12-18 explore, choose a real problem, build a project and present it at Demo Day. Explorers are 12-14, Builders are 15-18.

You help one student learn what they need for their own project and curiosity. Match the student's age group: short sentences and concrete everyday examples for Explorers; more depth and real tools for Builders. Be warm, encouraging and practical. Prefer doing over reading: small tasks the student can finish in 15-45 minutes with free tools, pen and paper or a phone.

Safety: keep everything age-appropriate. Never ask for or include personal data, contact details, or links to sign-up sites. Never suggest meeting strangers, sharing personal information online, spending money, or anything dangerous. If a question is about wellbeing, safety or something a trusted adult should handle, kindly suggest talking to their parent or mentor.

No variables are inserted.

5.2 Suggestion prompt (user message)

apps/web/src/lib/ai.ts L139-143 · schema L121-131

Suggest exactly 3 learning paths for this student, each different (e.g. one skill for their project, one thinking skill for their current step, one stretch idea). Do not repeat their earlier paths.

Student context:

```
${JSON.stringify(context, null, 2)}  
${interest ? `\nThe student says they are curious about: ${interest}` : ""}
```

```
SuggestionsSchema = { suggestions: [ {  
  title: string // "Short, inviting name for the learning path, under 60 characters"  
  goal: string // "One sentence: what the student will be able to do afterwards"  
  why: string // "One sentence linking it to their project or current step"  
} ] } // array described as "Exactly 3 suggestions"
```

Variables: `context` (`BloomContext` from `bloomContext()`) and `interest` (the form field, trimmed to 300 characters). The student's interest text is inserted as-is.

5.3 Plan generation prompt (user message)

apps/web/src/lib/ai.ts L172-178 · schema L148-159 · TASK_COUNT L163

Create a learning path called "`${path.title}`".
The student's goal: `${path.goal || "(not given - infer a sensible one from the title)"}`

Write exactly `${TASK_COUNT[path.depth]}` tasks in a sensible order: start with a "learn" task that explains the core idea simply, include at least one hands-on "do" task that moves their own project forward when possible, and finish with a "reflect" task asking what they learned and how they will use it.

Student context:

```
${JSON.stringify(context, null, 2)}
```

```
PlanSchema = {
  summary: string // "2-3 sentences, addressed to the student, on what this path covers and why it
helps them"
  tasks: [ {
    kind: "learn" | "do" | "reflect"
    title: string // "Short action title, under 70 characters"
    details: string // "What to do and how, as a short explanation plus clear steps. Plain text, 60-
180 words, no markdown headings"
  } ]
}
TASK_COUNT = { quick: "3", standard: "5", deep: "7" }
```

Variables: `title`, `goal` and `depth` from the form, and `context` as above. The interest text and the suggestion's `why` are **not** passed to this call.

5.4 Ask Bloom prompt

apps/web/src/lib/ai.ts L197–208

```
system: ${BLOOM_SYSTEM}

Answer the student's question in at most 150 words of plain text (no markdown headings or tables).
Explain with a simple example, then suggest one small next step. If the question is off-topic, answer
briefly if it is harmless and guide them back to their path. Don't just hand over finished work for a
task – help them think it through.

user: Learning path: ${topic.path}${topic.task ? `
Current task: ${topic.task}
${topic.taskDetails ??
""}` : ""}

Student context:
${JSON.stringify(context, null, 2)}

The student asks: ${question}
```

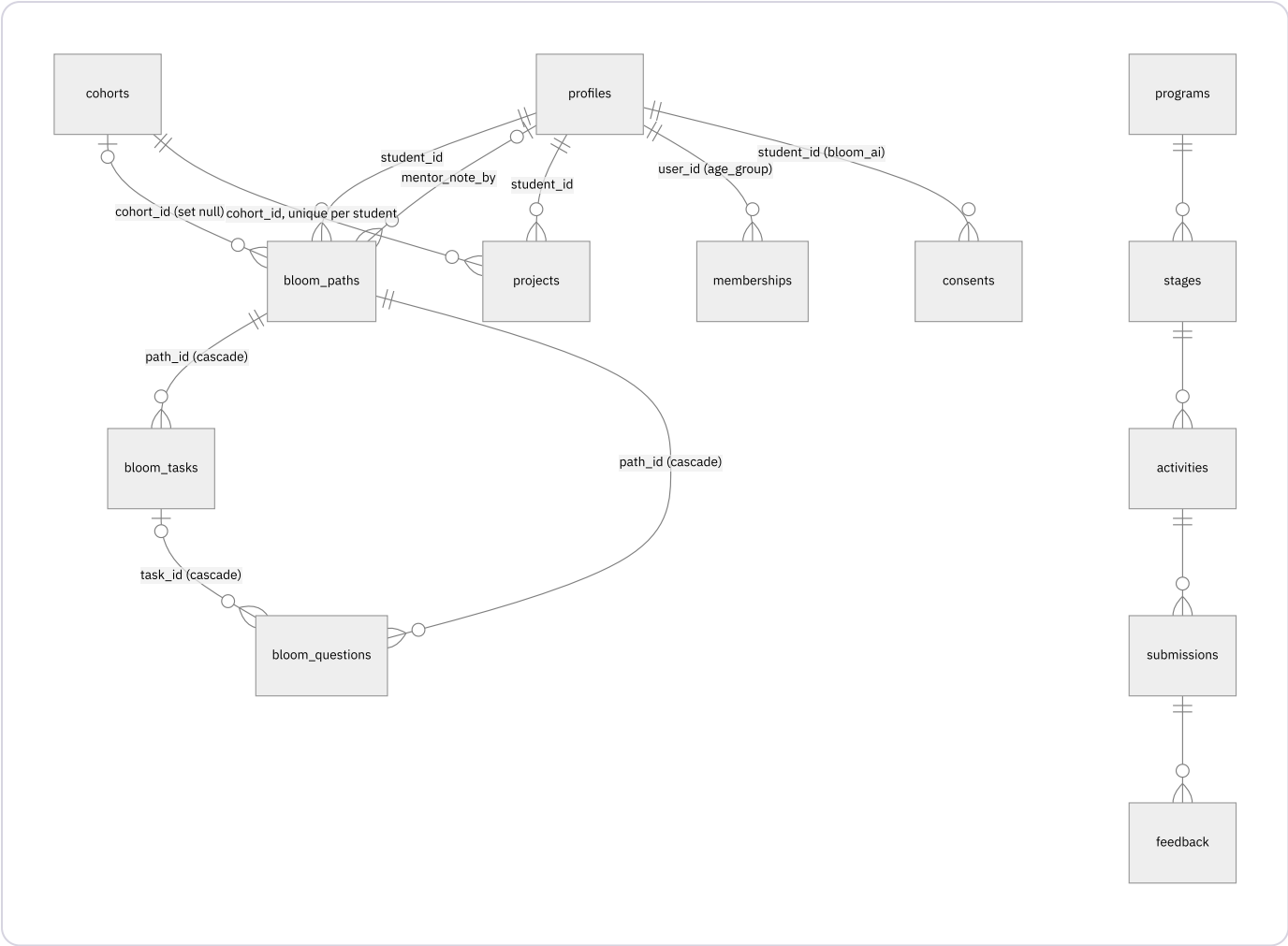
5.5 What the prompts say, topic by topic

TOPIC	COVERED?	WHERE AND HOW
Student age	Yes	The system prompt says 12–18, Explorers 12–14 and Builders 15–18, and to match the age group. The actual <code>ageGroup</code> is in the context. <code>profiles.birth_year</code> is not sent.
Project-based learning	Yes	8-week course: explore, choose a real problem, build, present at Demo Day. The plan prompt says a "do" task should "move their own project forward when possible".
Practical activities	Yes	"Prefer doing over reading", tasks of 15–45 minutes using free tools, pen and paper or a phone.
Evidence	No	Not mentioned.

TOPIC	COVERED?	WHERE AND HOW
Assessment	No	Not mentioned. The reflect task asks "what they learned and how they will use it".
Mentor involvement	Only for safety	"Suggest talking to their parent or mentor" for wellbeing or safety questions.
Project outcomes	Barely	Demo Day is named. No deliverable, milestone or success criteria are given.
Safety	Yes	No personal data, contact details or sign-up links, no meeting strangers or spending money, nothing dangerous; refer to a trusted adult.
Parents	Only for safety	As above.
Real-world activities	Implicit	Hands-on "do" tasks are asked for, while "meeting strangers" is forbidden. Talking to family is not addressed either way.
Language and locale	No	Prompts are English. <code>profiles.locale</code> is not sent, and stage names use <code>tr(..., "en")</code> .

06 Data model

Of the entities named in the request, these exist: student (`profiles`), project (`projects`), path (`bloom_paths`), step (`bloom_tasks`) and conversation-like Q&A (`bloom_questions`). `submissions` exists only for course activities, not for Bloom. These do **not** exist: a separate step-progress table, a separate notes table, conversations or messages as threads, and Bloom artifacts. **CONFIRMED**



Note that there is **no foreign key between bloom_paths and projects** , nor between Bloom and **stages** , **activities** or **submissions** . **stage_key** is a plain text copy of the stage key at creation time. **CONFIRMED**

bloom_paths migration L14-31

FIELD	TYPE	WRITTEN BY	NOTES
student_id	uuid → profiles	action	Owner. RLS insert requires auth.uid() and a student role.
cohort_id	uuid → cohorts, nullable	action	From the overview (most recent membership).
title	text 1-120	student, or AI via suggestion	Origin is not recorded.
goal	text ≤1000	student, or AI via suggestion	Empty when not given. An AI-inferred goal is not saved.
summary	text	AI (plan)	Empty without AI. Students can't update it (column grant).

FIELD	TYPE	WRITTEN BY	NOTES
stage_key	text, nullable	action	explore, choose, build or present for the current week.
depth	quick, standard or deep	student (route A always standard)	Controls the requested task count only.
status	enum active, completed or archived	action and auto-recompute	See section 7.
ai_generated	bool	action	True when the <i>plan</i> came from AI. It doesn't say whether the title or goal did.
mentor_note, _by, _at	text, uuid, timestamptz	mentor or admin RPC	One note per path, overwritten, max 2000 characters.
completed_at	timestamptz	trigger	Set when the status becomes completed and cleared otherwise.
created_at, updated_at	timestamptz	default and trigger	

bloom_tasks migration L35-48

FIELD	TYPE	WRITTEN BY	NOTES
path_id	uuid → bloom_paths, cascade	action	
student_id	uuid	trigger bloom_copy_student	Always overwritten from the path.
position	int	action	1..n for AI tasks; count+1 for added tasks. Not unique.
kind	enum learn, do or reflect	AI or student	Defaults to do.
title	text 1-160	AI or student	Origin is not recorded per task.
details	text ≤6000	AI or student	Shown as "How to do it".

FIELD	TYPE	WRITTEN BY	NOTES
status	enum todo, doing or done	student	
reflection	text ≤4000, nullable	student	Holds both "Add a note" and reflect answers.
completed_at	timestampz	trigger	Set on done and cleared otherwise.

bloom_questions migration L54–62

path_id (cascade), task_id (nullable, cascade), student_id (trigger), question (1–1000 characters, student), answer (AI text), created_at . Rows can't be updated by students (revoke update), and there is no delete policy for students.

Related rules

- Read access:** can_view_student(student_id) covers the student, linked parents, the student's mentors and admins. **Writes** are by the student only, plus admins. Students may update only title, goal, status on paths and title, details, status, reflection, position on tasks (column grants, L151–166).
- Audit:** write_audit_log records actor, action, table, id and changed column names, never values (20261004120000_foundation.sql L479–507).
- Export:** export_student_data includes all three Bloom tables (migration L204–237).

Simple form

Student (profiles)

- └ Membership (cohort, age_group)
- └ Project (one per cohort: area, title, problem, status) ← not linked to Bloom
- └ Course submissions → feedback (per programme activity) ← not linked to Bloom
- └ BloomPath (cohort_id?, stage_key, mentor_note)
 - └ BloomTask (kind, status, reflection)
 - └ BloomQuestion (question, answer, task_id?)

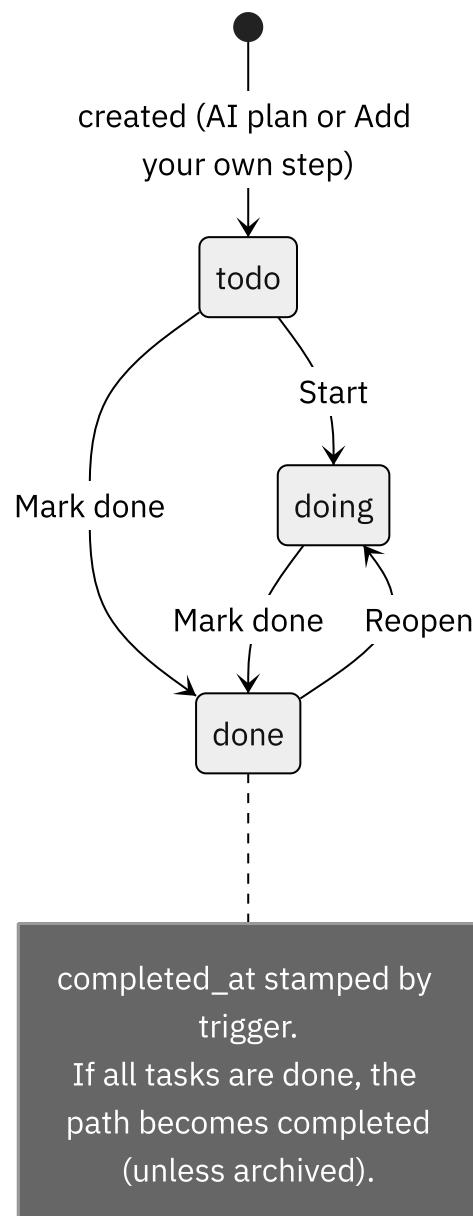
Points to watch

TOPIC	HOW IT WORKS TODAY
Step status	Column bloom_tasks.status . There is no history table; the audit log records only that status changed.
Mark done	updateTask sets status='done' . The trigger stamps completed_at , then the path status is recomputed.

TOPIC	HOW IT WORKS TODAY
Completion %	Computed in code, not stored. Per path it is <code>round(done/total×100)</code> (<code>components/course.tsx</code> <code>ProgressBar</code> L29–30, with <code>done</code> and <code>total</code> from <code>loadBloomPaths</code>). The Bloom home "overall" figure counts tasks across <i>all</i> paths, including archived (<code>bloomStats</code> , page L41).
Evidence and submissions	None for Bloom.
Notes tied to steps	Yes, one <code>reflection</code> per task.
AI assessments	None.
Project artifacts	None in Bloom. Course submissions can have files (<code>submission_files</code> , storage bucket <code>submissions</code>).
Student answers persisted	Only as <code>reflection</code> text and <code>bloom_questions.question</code> .

07 Step lifecycle

Example: "Ask your uncle 5 simple questions", a `do` task at position 2 in the example path. Its stored content couldn't be read (see section 12). The lifecycle below applies to every task. **CONFIRMED** from code.



1. **Generation:** one element of `PlanSchema.tasks` from `planBloomPath`, inserted by `createPath` (L90–102).
2. **Fields:** `id`, `path_id`, `student_id`, `position`, `kind`, `title`, `details`, `status=todo`, `reflection=null`, `completed_at=null` and timestamps.
3. **Rendering** (`BloomPathDetail`, `components/bloom.tsx` L91–127): a number circle (a check mark when done), a kind badge, a status badge and the title. A "How to do it" `<details>` shows the details text and is open unless the task is done. Then come any Q&A for this task, then `TaskControls`, then an "Ask Bloom about this step" toggle (only with consent and AI configured).
4. **Start:** posts `status=doing` to `updateTask`. The task updates (`.eq("student_id")` plus RLS), sibling statuses are re-read and the path status recomputed, then `refresh()`.
5. **Mark done:** posts `status=done` and the optional `reflection` textarea, which is visible only for reflect tasks or after "Add a note". The result message is "taskDone", or "pathDone" when it was the last task.
6. **Evidence required:** none. The reflect textarea is not `required`.
7. **AI checks:** none.
8. **Answer or submission required:** none. Answers to the uncle's questions can only go into the free-text note.
9. **Mentor involvement:** none in the transition. A mentor can later read the path and leave one note on it.

10. **Stored:** `status` , `reflection` (or null), `completed_at` (trigger) and `updated_at` . The path's `status` and `completed_at` may also change, and an audit row is written.
11. **Percentage:** recomputed when the page reloads, as `done ÷ total`. Each task carries equal weight.

Path status machine: `active ⇌ completed` is automatic, driven only by `updateTask` . `archived` is set and cleared manually through `setPathStatus` (Archive and Restore buttons). Restore always sets `active` , even if every task is done. Delete removes the path, cascading to its tasks and questions, after a browser `confirm()` . The backend `setPathStatus` also accepts `completed` , but the UI never sends it. **CONFIRMED**

08 Ask Bloom flow

CLICK

"Ask Bloom about this step" opens a compact `AskForm` under the task with a hidden `taskId` . A general form also sits at the bottom of the page without a `taskId` .

ACTION

`askQuestion` (actions.ts L182–208) checks the student role, a non-empty question (max 1000 characters) and `bloom_ai` consent.

CONTEXT

In parallel it loads the path's `title` with its tasks' `id`, `title`, `details` , and runs `aiContext()` . It then finds the task by id within the path; an unknown id is treated as a general question.

LLM

`askBloom()` makes one call to `beta.messages.create` with the system prompt from 5.4 and returns plain text.

PERSIST AND RENDER

Inserts into `bloom_questions` (question, answer, `task_id`), then `refresh()` . `QuestionList` shows the pair under the task or in the general section, and it is visible to parents, mentors and admins.

DOES BLOOM KNOW OR DO THIS?	ANSWER
Current project	Yes, through <code>context.project</code> (area, title, problem, status).
Current path	Title only. Not the goal or summary.
Current step	Title and details, when asked from a task.
Previous steps in the path	No. The other tasks are loaded but not sent.
Previous student answers or notes	No.
Chat history	Stored in <code>bloom_questions</code> but never sent back, so every question is answered without memory.
Change the path, mark steps complete, create steps	No. No tools are given and the reply is only stored as text.

DOES BLOOM KNOW OR DO THIS?	ANSWER
Affects project state	No.
Other paths	Up to 12 titles through <code>previousPaths</code> , which includes the current path's own title.

09 "Add your own step" flow

- **Who creates it:** the student, entirely manually (`AddTaskForm` to `addTask` , `actions.ts` L107–127). No AI is involved.
- **Fields:** `title` is required (≤ 160 characters). `kind` comes from a select of learn, do or reflect, defaulting to do; `oneOf()` enforces it in code and a Postgres enum in the database. `details` is optional (≤ 6000 characters).
- **Persistence:** insert with `position = count(tasks in path) + 1` . Ownership comes from the trigger that copies `student_id` from the path, plus the RLS check `student_id = auth.uid()` ; the action itself doesn't verify the path.
- **Origin:** indistinguishable from AI tasks in the database. There is no per-task origin flag, and `ai_generated` is per path.
- **Does Bloom know about it?** Only when the student asks Bloom about that specific task. Suggestion and plan calls see path titles only.
- **Progress:** counts exactly like any other task. However `addTask` doesn't recompute the path status, so adding a step to a `completed` path leaves it completed with an open task until another task is updated. **CONFIRMED**
- **Editing and deleting steps:** no UI or action exists, although column grants and an RLS delete policy would allow it. The UI copy "you can add or change steps later" (`bloom.setBloomPlan`) promises more than the UI offers.

10 Project memory and context

There is no persistent "project memory" object. Bloom's knowledge is rebuilt on every call from live rows by `bloomContext()` .

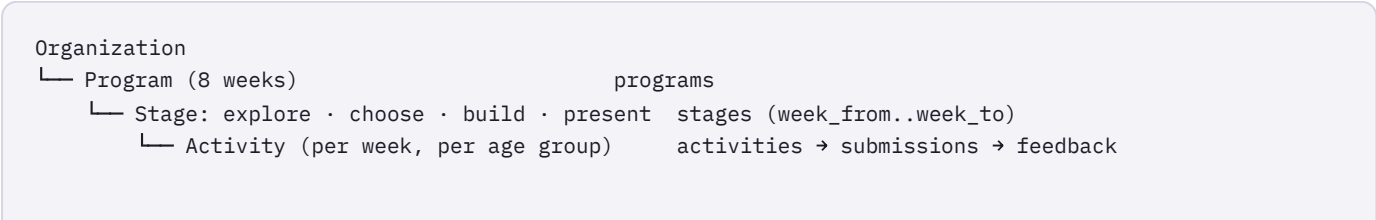
CONTEXT ITEM	SOURCE → STORAGE	SENT TO BLOOM?
Original idea and problem	Student project page (<code>saveProject</code> , <code>app/student/actions.ts</code> L81) → <code>projects.title</code> , <code>problem</code>	Yes, as <code>context.project</code>
Project goal or deliverable	No such field	n/a
Selected track	<code>projects.area</code> (technology, design, business, <code>social_impact</code> or undecided)	Yes
Project status	<code>projects.status</code> (exploring, chosen, building, presenting or done)	Yes
Interests	The "curious about" field. Not stored.	Suggestion call only, once
Age group	Admin sets it on <code>memberships.age_group</code>	Yes

CONTEXT ITEM	SOURCE → STORAGE	SENT TO BLOOM?
Age or birth year	<code>profiles.birth_year</code>	No
Current milestone	<code>stages</code> (seed: explore weeks 1–2, choose 3–4, build 5–6, present 7–8) plus <code>cohorts.start_date</code> → <code>currentWeek()</code>	Stage name and week number. The week is clamped to 1–8, so before the start the student is reported as in week 1 (Explore).
Stage summary and this week's activities	<code>stages.summary</code> , <code>activities.title</code> and <code>instructions</code>	No, even though they are loaded
Research, previous work, decisions	Course <code>submissions</code> (body, link, files) per activity	No (the progress-card AI does receive them)
Mentor feedback	<code>feedback</code> on submissions; <code>bloom_paths.mentor_note</code> ; <code>progress_cards</code>	No
Bloom history	<code>bloom_paths</code> , <code>bloom_tasks</code> , <code>bloom_questions</code>	Path titles only (≤12)
Reflections and notes	<code>bloom_tasks.reflection</code>	No
Learning outcomes and skills	No such data	n/a
Locale, country	<code>profiles.locale</code> , <code>country</code>	No
Class sessions	<code>sessions</code>	No

Only one cohort is considered: `loadStudentOverview` picks the most recent student membership. A student with no membership gets `context = {ageGroup:null, step:null, week:null, project:null, previousPaths:[...]}` . **CONFIRMED**

11 Learning-path and course hierarchy

What a path represents: option **E**, a **combination**. It is a short, student-chosen learning pathway (C), written by AI as a flat task list (D), sometimes nudged toward project work. It is not a traditional course (A): there are no lessons or content beyond the task text. It is not a project plan (B) either: there are no milestones, deliverables or dependencies, and no link to the project. **INFERRED** from the schema and prompts.



Student, within a Cohort (an instance of a Program)	
└ Project (1 per cohort)	projects
└ Bloom path (many, any time)	bloom_paths ← tagged with cohort_id + stage_key only
└ Task (flat, ordered)	bloom_tasks
└ Question and answer	bloom_questions

- **What causes a new path:** only a student click (section 3). Nothing is automatic or scheduled.
- **Does Bloom know the 8-week journey?** Only as the one-sentence description in `BLOOM_SYSTEM` plus the current step name and week.
- **Does it plan all 8 weeks?** No. It plans one standalone path of 3, 5 or 7 tasks per request.
- **Do later paths depend on earlier work?** Only to the extent that earlier titles are listed to avoid repeats. There are no prerequisites or ordering between paths.
- **Explicit final deliverable:** none in the data model. "Demo Day" appears in the prompt, and "portfolio page" appears only in the seed stage summary, which isn't sent.
- **How Bloom decides what's next:** it doesn't. The student home "Continue" card is deterministic: the most recently created active path, and its first not-done task by position (`app/student/page.tsx` L27–28).

12 One complete generation trace: "How a Small Business Really Works"

Original model input and output: unavailable, by design. The app never logs prompts or responses (section 4), and the audit log stores only column names. Even with database access, the only surviving AI output is what was saved: `summary` and each task's `title` and `details`. The suggestion response (including its `why`) was never saved anywhere. **UNAVAILABLE**

The saved rows themselves couldn't be read in this session (database query blocked). The values below come from the request text, with everything else shown as placeholders.

Input to the AI (reconstructed template)

The likely route is A (suggestion, then "Start this path"). **INFERRED** Reasons: there are 5 tasks, and suggestion cards always submit `depth=standard`. The goal is phrased "You will be able to...", which matches the suggestion schema's description "what the student will be able to do afterwards". And the uncle's shop must have come from free text, either the interest field or the project problem. Route B with a student-written goal and standard depth would produce the same rows, so this can't be settled from code.

Call 1 (`suggestBloomPaths`), user message:
Suggest exactly 3 learning paths for this student, each different (...). Do not repeat their earlier paths.

Student context:

```
{
  "ageGroup": "<explorer|builder>",          ← memberships.age_group      [not read]
  "step": "<Explore|Choose|Build|Present>",    ← stage for current week      [not read]
  "week": "<1-8>",
  "project": { "area": "<...>", "title": "<...>", "problem": "<...>", "status": "<...>" } | null,
  "previousPaths": [ <earlier titles> ]
}
```

The student says they are curious about: `<interest text>` [not stored anywhere]

Call 2 (`planBloomPath`), user message:

Create a learning path called "How a Small Business Really Works".

The student's goal: You will be able to draw a simple map of how your uncle's shop gets customers,

sells things and makes money.

Write exactly 5 tasks in a sensible order: start with a "learn" task (...), include at least one hands-on "do" task (...), and finish with a "reflect" task (...).

Student context: { ...same object... }

For reference, the screenshot shared earlier in this session showed this interest text typed into the field: "...ed to build my own successful business. I need to know how a successful business work. and how can I start one of my own". It may or may not be the input that produced this path. **INFERRED**

Structured AI response (shape; content only partly known)

```
{
  "summary": "<2-3 sentences: saved in bloom_paths.summary, not read>",
  "tasks": [
    { "kind": "learn",    "title": "The 3 big parts of any small shop",    "details": "<not read>"
  },
    { "kind": "do",      "title": "Ask your uncle 5 simple questions",    "details": "<not read>"
  },
    { "kind": "do",      "title": "Draw your shop money map",            "details": "<not read>"
  },
    { "kind": "do",      "title": "Spot where a logo would help customers", "details": "<not read>"
  },
    { "kind": "reflect", "title": "What did I learn about the shop?",      "details": "<not read>"
  }
  ]
}
```

Saved records

```
bloom_paths: title="How a Small Business Really Works", goal="You will be able to draw...",
              depth="standard", ai_generated=true, status="active", stage_key=<step at creation>,
              cohort_id=<cohort>, summary=<AI summary>
bloom_tasks: 5 rows, position 1.5, kinds learn/do/do/do/reflect, status "todo" at creation
```

Why these five steps (explained from the prompt)

- **5 tasks:** `TASK_COUNT.standard = "5"`.
- **Learn first, Reflect last:** the plan prompt requires it.
- **Three Do tasks in between:** the prompt requires "at least one hands-on 'do' task", and the system prompt says "prefer doing over reading".
- **Interviewing a relative:** fits "pen and paper or a phone" and the ban on meeting strangers. The goal itself names the uncle's shop.
- **"Spot where a logo would help customers":** fits "moves their own project forward when possible". This suggests the student's project mentions a logo or the design area. **INFERRED**; the project row wasn't read.
- **"What did I learn about the shop?":** the prompt's required reflect task ("what they learned and how they will use it").

What the student sees

On the path page: header with the Active badge, the stage badge and "Standard", the path title, and the goal as the description. Then the summary, a progress bar showing "0 of 5 steps done" and 0%, and five task cards, each with a Learn, Do or Reflect badge, a To do

badge, the title, an open "How to do it" section, Start and Mark done buttons ("Not finished? Reopen" once done), "Add a note" (not on the reflect task, which shows its textarea), and "Ask Bloom about this step". Below them come "Ask Bloom" and "Add your own step".

13 Current capabilities matrix

CAPABILITY	EXISTS?	WHERE	AI OR DETERMINISTIC	PERSISTED?	NOTES
Path generation	Yes	<code>suggestBloomPaths</code> , <code>planBloomPath</code> , <code>createPath</code>	AI (titles from AI or student)	Path yes; suggestions no	Only when the student asks.
Step generation	Yes	<code>planBloomPath</code>	AI, all at once	Yes	Never regenerated or added to by AI later.
Student progress	Yes	<code>updateTasks</code> , <code>loadBloomPaths</code>	Deterministic	Status yes; % computed	Self-reported.
Mark done	Yes	<code>TaskControls</code> → <code>updateTask</code>	Deterministic	Yes	No requirements.
Notes	Yes	<code>bloom_tasks</code> <code>.reflection</code>	Student	Yes	One per task, overwritten.
Chat with Bloom	Partly	<code>askQuestion</code> → <code>askBloom</code>	AI	Yes	Single question and answer, no memory.
Quizzes	No	—	—	—	
Student answers	Partly	<code>reflection</code>	Student	Yes	Free text only.
File upload	No (Bloom)	Course submissions only	—	—	<code>submission_files</code> isn't used by Bloom.
Photo upload	No (Bloom)	As above	—	—	
Voice input	No	—	—	—	

CAPABILITY	EXISTS?	WHERE	AI OR DETERMINISTIC	PERSISTED?	NOTES
Drawing or canvas	No	—	—	—	"Draw your money map" is done off-platform.
Evidence submission	No (Bloom)	Course submissions only	—	—	
AI assessment	No	—	—	—	
Rubric assessment	No	—	—	—	
Mentor approval	No	—	—	—	Only a free-text <code>mentor_note</code> per path. Course submissions have mentor review instead.
Project artifacts	No	—	—	—	
Project memory	Partly	<code>bloomContext</code>	Deterministic	Live rows, no memory store	See section 10.
Adaptive learning	Minimal	Prompts	AI	No	Age-group tone and avoiding repeated titles.
Prerequisite logic	No	—	—	—	Tasks can be done in any order.
Revision and resubmission	Reopen only	<code>updateTask</code> done → doing	Deterministic	Latest state only	No history.
Skill tracking	No	—	—	—	

CAPABILITY	EXISTS?	WHERE	AI OR DETERMINISTIC	PERSISTED?	NOTES
Learning outcome tracking	No	—	—	—	The path <code>goal</code> is never checked.
Portfolio building	No (Bloom)	—	—	—	A <code>public_portfolio</code> consent and <code>projects.is_public</code> exist, unrelated to Bloom.
Visibility for others	Yes	RLS <code>can_view_student</code> ; mentor, parent and admin pages; <code>Timeline</code>	Deterministic	Yes	Parents need no extra consent to read.
Archive and delete	Yes	<code>setPathStatus</code> , <code>deletePath</code>	Deterministic	Yes	Delete is a hard cascade.

14 Important files and their responsibilities

FILE	RESPONSIBILITY
<code>apps/web/src/lib/ai.ts</code>	Model constant, all prompts, Zod schemas, Anthropic calls and the error reasons.
<code>apps/web/src/app/student/bloom/actions.ts</code>	All student Bloom server actions and AI context assembly.
<code>apps/web/src/app/student/bloom/forms.tsx</code>	Client forms: <code>StartPath</code> , <code>TaskControls</code> , <code>AddTaskForm</code> , <code>AskForm</code> , <code>PathActions</code> .
<code>apps/web/src/app/student/bloom/page.tsx</code>	Bloom home: stats, start card, path lists.
<code>apps/web/src/app/student/bloom/[id]/page.tsx</code>	Student path page.

FILE	RESPONSIBILITY
<code>apps/web/src/components/bloom.tsx</code>	Shared renderers: path card, detail, Q&A list, timeline, last-active badge, compact list.
<code>apps/web/src/lib/data/bloom.ts</code>	Reads, consent check, stats, AI context builder, timeline, totals per student.
<code>apps/web/src/lib/data/overview.ts</code>	Student course overview, the source of Bloom's context.
<code>apps/web/src/lib/programme.ts</code>	<code>currentWeek</code> , <code>stageForWeek</code> , <code>tr</code> .
<code>supabase/migrations/20261005140000_bloom.sql</code>	Bloom schema, triggers, RLS, mentor-note RPC, export.
<code>supabase/seed.sql</code>	Programme and stage definitions (8 weeks, 4 stages).
<code>apps/web/scripts/seed-demo.mjs</code> L109–145	Demo consents and one demo path, "Talking to people about a problem", with 3 tasks.
<code>supabase/tests/rls.test.sql</code> L259–323	pgTAP tests for Bloom visibility, ownership, the mentor note and completion stamping.
<code>apps/web/src/app/mentor/actions.ts</code> , <code>mentor/forms.tsx</code> , <code>mentor/groups/[cohortId]/students/[studentId]/bloom/[pathId]/page.tsx</code>	Mentor read view and note.
<code>apps/web/src/app/parent/children/[id]/bloom/[pathId]/page.tsx</code> , <code>parent/actions.ts</code>	Parent read view; <code>bloom_ai</code> consent toggle.
<code>apps/web/src/app/student/page.tsx</code>	Home "Continue" card and timeline.
<code>apps/web/messages/en.json</code> (<code>bloom.*</code>)	All UI copy, including the AI error messages.

15 Confirmed gaps in understanding, and risks found

Could not trace

- **UNAVAILABLE** The content of real paths in the hosted database, including the example path's summary, task details, project row and interest text.
- **UNAVAILABLE** Original AI requests and responses, which are never logged.
- **UNAVAILABLE** Which model actually served each request when `fallbacks: "default"` is set.
- **UNAVAILABLE** Whether the Bloom migrations are applied on the hosted project, and the RLS test results (no local Supabase was running).
- **INFERRED** How `betaZodOutputFormat` converts the `.describe()` text. I assume the descriptions reach the model as JSON-schema descriptions and that only structure is enforced; I didn't read the SDK source.

Code behaviour where the frontend and backend differ, or logic conflicts **CONFIRMED** by reading; not reproduced

- **A note can be wiped.** `updateTask` always writes `reflection: reflection || null`. For a reopened non-reflect task, the textarea is hidden until "Add a note" is clicked. Clicking "Mark done" then sends no reflection and clears the saved note.
- **Path and tasks are inserted separately.** If the task insert fails, the path is kept with no tasks and only a server log entry.
- **Counts and lengths aren't enforced.** Task count, suggestion count and length limits are prompt hints, not schema rules. Up to 10 tasks are accepted regardless of the chosen depth.
- **An inferred goal is discarded.** When the goal is blank the model is told to infer one, but there is no field to return it in.
- **The suggestion's `why` is shown and then lost.** It isn't saved or passed to the plan call.
- **Path status can go stale.** `addTask` doesn't recompute it, so a completed path can hold open tasks. Restoring an archived path always sets `active`, even when every task is done.
- **The backend accepts more than the UI sends.** `setPathStatus` accepts `completed` with no task check. `updateTask` accepts any status from any state, including `todo`.
- **Unused permissions.** Task `title`, `details` and `position` update grants and the task delete policy exist with no UI, and the UI copy says steps can be changed later.
- **Totals disagree.** Bloom home "overall %" and the mentor group totals (`loadBloomTotals`) count tasks of archived paths, while the mentor path count excludes archived paths.
- `previousPaths` includes archived paths and, in Ask Bloom, the current path itself.
- **Week before the start.** Before the cohort starts (week 0), the context reports week 1 and Explore, and `stage_key` is saved as `explore`.
- **No limits on archived paths.** Ask Bloom and task updates work on archived paths.
- **Duplicate positions are possible.** `position` for added tasks is count+1 and not unique, so concurrent adds can collide.
- `ai_generated` is ambiguous. It marks AI-planned tasks, not AI-written titles or goals, and per-task origin isn't recorded.

Dead code, duplicates, unused fields

- `askBloom` and `draftProgressCard` repeat the same create, refusal, text-join and error-mapping logic instead of sharing `structured()`. This is a duplication note, not a defect.
- The tasks loaded in `askQuestion` (all `bloom_tasks` with details) are used only to find one task; the rest are discarded.
- The `loadStudentOverview` fields used by Bloom are a small subset: activities, submissions, sessions and stats are loaded and then ignored for AI context.
- No unused Bloom columns were found. Every column is written or read somewhere.

16 Questions that still need investigation

1. What are the saved `summary` and task `details` for "How a Small Business Really Works", and what are that student's `projects` row and `age_group` ? This needs a database read, which is blocked here.
2. Was that path created through route A (suggestion) or route B (own topic)? The audit log can't tell. If the goal text matches the suggestion style word for word, route A is more likely.
3. How often does the model return a task count other than the requested 3, 5 or 7? Nothing records it.
4. Which model actually answers under `fallbacks: "default"` , and what does each call cost? There is no usage logging.
5. Are the three Bloom migrations applied on the hosted project, and do the pgTAP tests pass?
6. Does the product intend Bloom paths to belong to the project (there is no `project_id`) or to the student? The code does the latter.
7. Should parents see Bloom content without a separate consent? Today the RLS lets them, and the consent text says "you and their mentor can read every question and answer".